

소프트웨어공학

Homework #3-2

2019060546

주원웅

1. Decorator

런타임 시점에 기능을 동적으로 추가 가능

데코레이터 패턴의 핵심은 "다른 객체를 생성자의 인자로 받아와 그 객체를 감싸면서 기능을 확장하는 것이다"

공통 인터페이스나 부모 클래스를 통해 기존 객체와 동일한 인터페이스를 유지하면서 새로운 기능을 추가한다.

CaesarCipher

1. 알파벳을 3개만큼 왼쪽으로 밀어내는 암호화 method
2. 3개만큼 다시 오른쪽으로 가져오는 복구를 하는 method

두 method 모두 static 이므로 바로 호출하여 사용 가능.

EncryptWriter

- Writer 클래스의 기능을 확장한다. Write 함수 호출 시 CaesarCipher 의 암호화 method 를 호출하여 input 에 대해 3칸 밀어내고 write 를 진행하도록 한다.

Writer의 write 함수에 위 기능을 추가.

DecryptReader

- Reader 클래스의 기능을 확장한다. Read 함수 호출 시 CaesarCipher 의 암호복구 method 를 호출하여 input 에 대해 3칸 당겨오고 Read 를 진행하도록 한다.

Reader의 Read 함수에 위 기능을 추가.

2. Strategy

Strategy 패턴은 어떤 동작을 캡슐화하고, 런타임에 원하는 알고리즘을 동적으로 선택할 수 있도록 하는 디자인 패턴이다.

여러 알고리즘을 인터페이스로 정의하며, 실행 시 동작(전략)을 설정하여 동작 방식을 변경한다.

EncryptionStrategy

암호화와 복호화를 처리하는 클래스들의 인터페이스.

CaesarCipherStrategy

Caesar Cipher 알고리즘을 구현한 클래스.

인터페이스가 **EncryptionStrategy** 이다.

CasInvertStrategy

소문자를 대문자로, 대문자를 소문자로 변환하는 간단한 암호화, 복호화가 존재하는 클래스.

인터페이스가 **EncryptionStrategy** 이다.

EncryptWriter와 DecryptReader의 Strategy 패턴 적용

EncryptWriter와 DecryptReader는 Strategy 객체(Caesar Cipher or CasInvertStrategy)를 생성자에서 동적으로 받아와 각 클래스에 알맞게 암호화 알고리즘을 설정할 수 있다.

이를 통해 다양한 알고리즘을 쉽게 확장하거나 교체할 수 있다.

3. Visitor

Visitor 패턴은 객체 구조를 변경하지 않고, 구조 내의 각 요소에 대해 새로운 동작을 할 수 있도록 해주는 패턴이다.

구조가 복잡하거나 고정되어 있는 시스템에서 다양한 동작을 추가하고 관리할 때 적합하다.

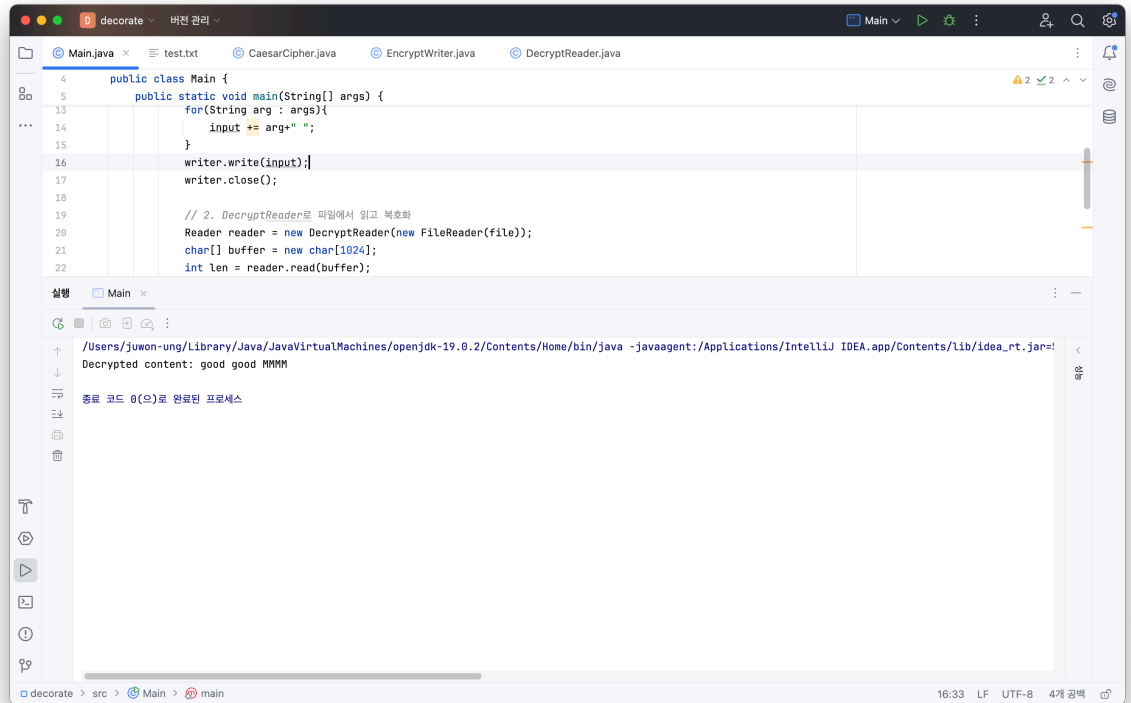
PrintVisitor를 조금 수정하여 Find Visitor를 만들었다.

위 3가지 모두 CommanLine 을 통해 인자를 받아와 실행하도록 만들었다.

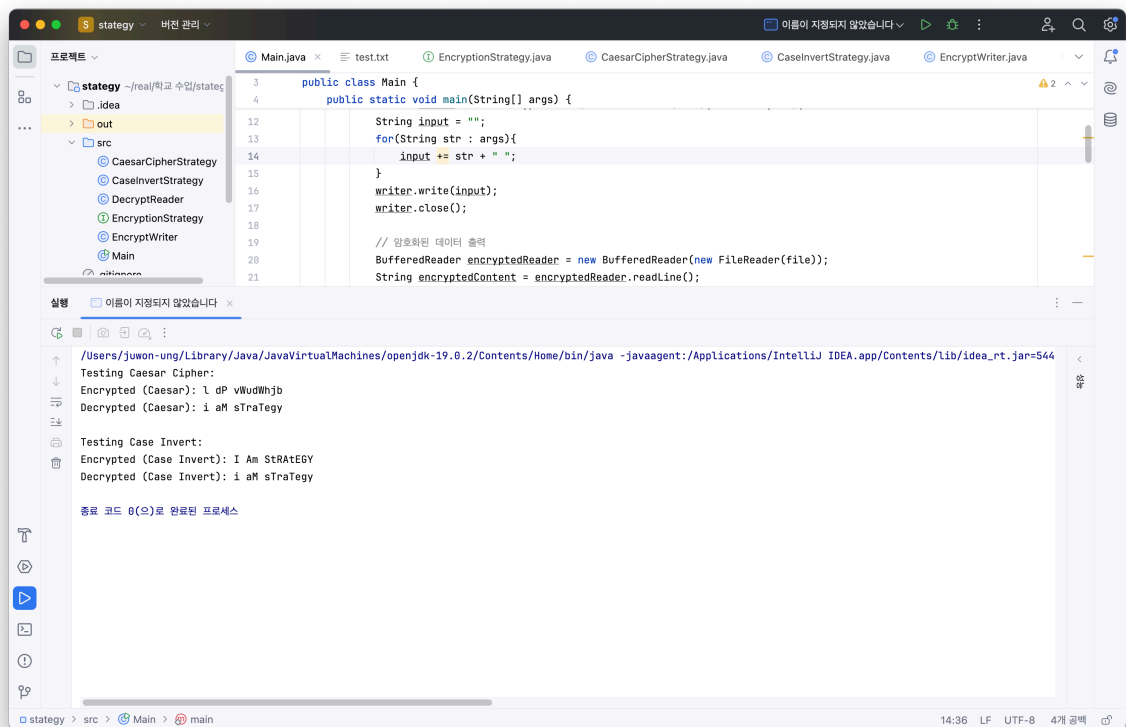
Intellij 를 사용하여 작성하였으며 결과는 다음과 같다.

실행결과

1. Decorator



2. Strategy



3. Visitor

